

TABLE OF CONTENTS

Flutter Bottom Navigation Bar with Stateful Nested Routes using GoRouter	1
Stateful Nested Navigation with GoRouter	4
GoRouter Implementation with StatefulShellRoute	6
Where the magic happens: StatefulNavigationShell	10
Stateful Nested Navigation: Summary	12
Making the UI responsive with NavigationRail and LayoutBuilder	14
Creating a Scaffold with NavigationBar Widget	15
Creating a Scaffold with NavigationRail Widget	16
Updated Scaffold with Nested Navigation	18
Source Code	20
Answering Common Questions	20
Does Stateful Nested Navigation work on Flutter web?	21
Does “pop-to-root” work?	21
What About the Back Button on Android?	23
Can I Navigate Programmatically to Another Branch from a Nested Route?	24
Are there some Unexpected Issues with GoRouter?	25
Which Routing Package Should You Use?	26
Conclusion	26
Flutter Foundations Course Now Available	27
Flutter Foundations Course	27
Want More?	28
Flutter In Production	28
Flutter & Firebase Masterclass	28
The Complete Dart Developer Guide	28
Flutter Animations Masterclass	28



Flutter Bottom Navigation Bar with Stateful Nested Routes using GoRouter

#dart

#flutter

#navigation

#gorouter



Andrea Bizzotto

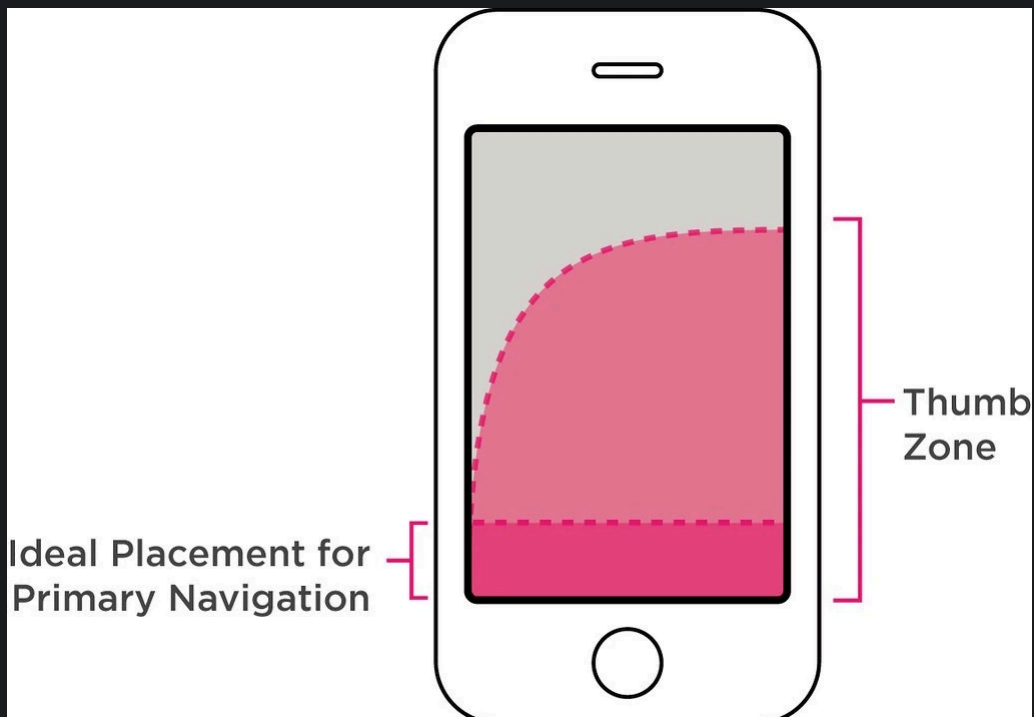
Updated Jun 19, 2023 · 16 min read



Source code on GitHub

One of the most popular UX patterns on mobile is to show a **bottom navigation bar with tabs**.

With this pattern, all the tabs are within the "thumb zone" where they can be easily reached:



Thumb zone on mobile devices. Credit: [Rosenfield Media](#)

But how can we implement bottom navigation in a Flutter app?

As it turns out, the `NavigationBar` widget gives us a convenient way of switching between the primary destinations (aka tabs) in our app.

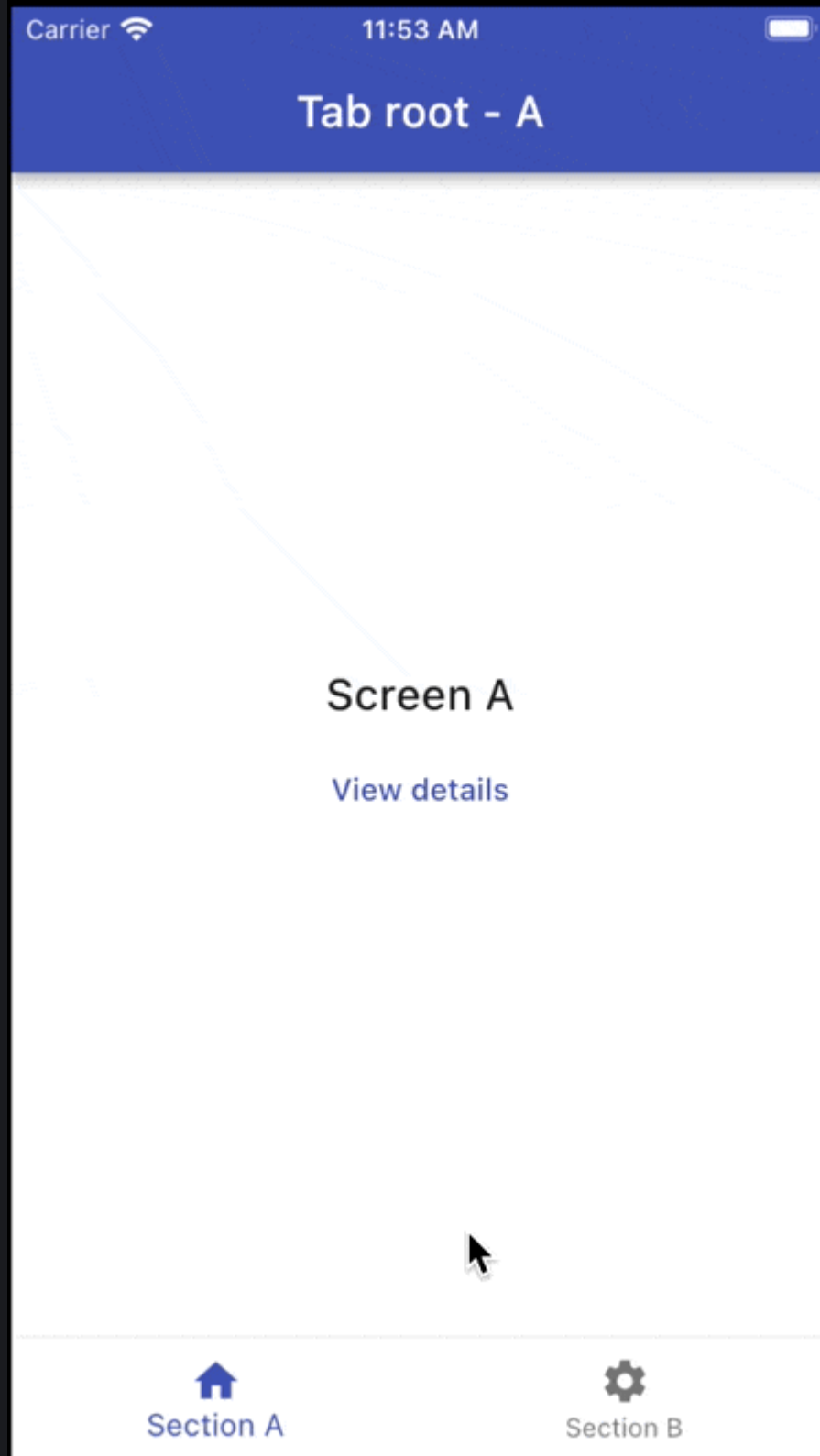
And while `NavigationBar` takes care of showing the bottom navigation UI, it doesn't handle any state or routing logic:

```
Scaffold(  
  body: /* TODO: decide which page to show, depending on the selected  
index */,  
  bottomNavigationBar: NavigationBar(  
    selectedIndex: /* TODO: where does this come from? */,  
    destinations: const [  
      // the appearance of each tab is defined with a  
[NavigationDestination] widget  
      NavigationDestination(label: 'Section A', icon: Icon(Icons.home)),  
      NavigationDestination(label: 'Section B', icon:  
Icon(Icons.settings)),  
    ],  
    onDestinationSelected: (index) { /* TODO: move to the tab at index */  
},  
  ),  
)
```

And once we dive a bit deeper, some other questions arise:

- How to show different pages when we switch between tabs?
- How to perform nested navigation and push additional pages **inside** each tab?
- How to preserve the navigation state of each tab?

In other words, how can we implement this behaviour?



Stateful nested navigation example. Note how the counter value is preserved when switching between tabs.

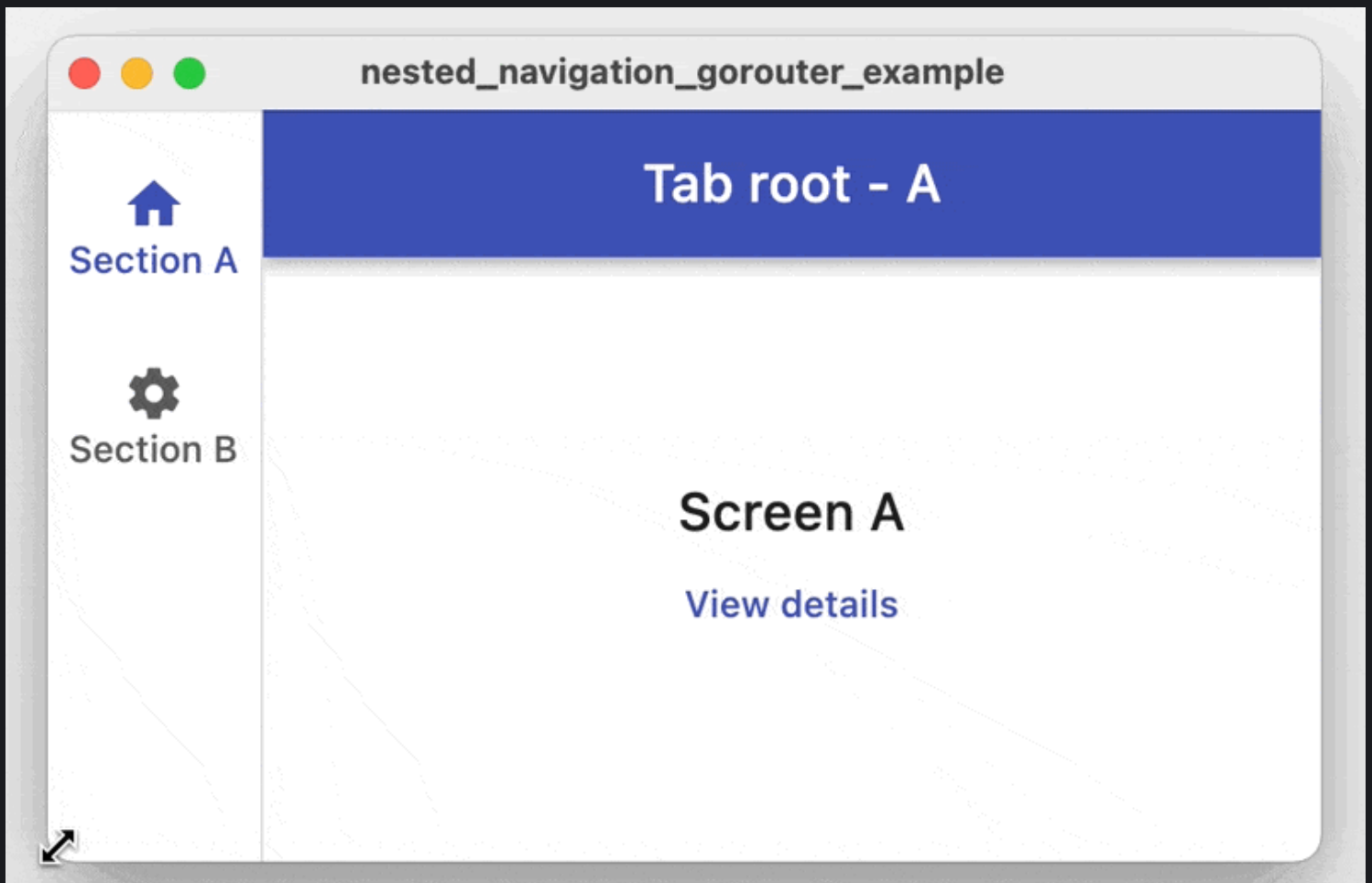
I've already tackled these questions in a [previous article](#), showing how to implement nested navigation with a combination of `Stack`, `Navigator`, and `Offstage` widgets. However, my previous solution had some limitations, and since it was built with the Navigator 1.0 APIs, it didn't support **deep linking** and **navigation by URL**.

But hold on! We now have powerful routing libraries such as [GoRouter](#), [Beamer](#), and [AutoRoute](#), which are built on top of the Flutter Navigator 2.0 APIs.

And as of June 2023, all these libraries support **stateful nested navigation**.

So in this article, I'll show you how to use the latest GoRouter APIs (`StatefulShellRoute`, `StatefulShellBranch`, `StatefulNavigationShell`) to implement **stateful nested navigation** in your Flutter apps.

And once all the essential routing code is in place, I'll also show you how to create a responsive UI, so we can switch between `NavigationBar` and `NavigationRail` depending on the window size, like in this example:



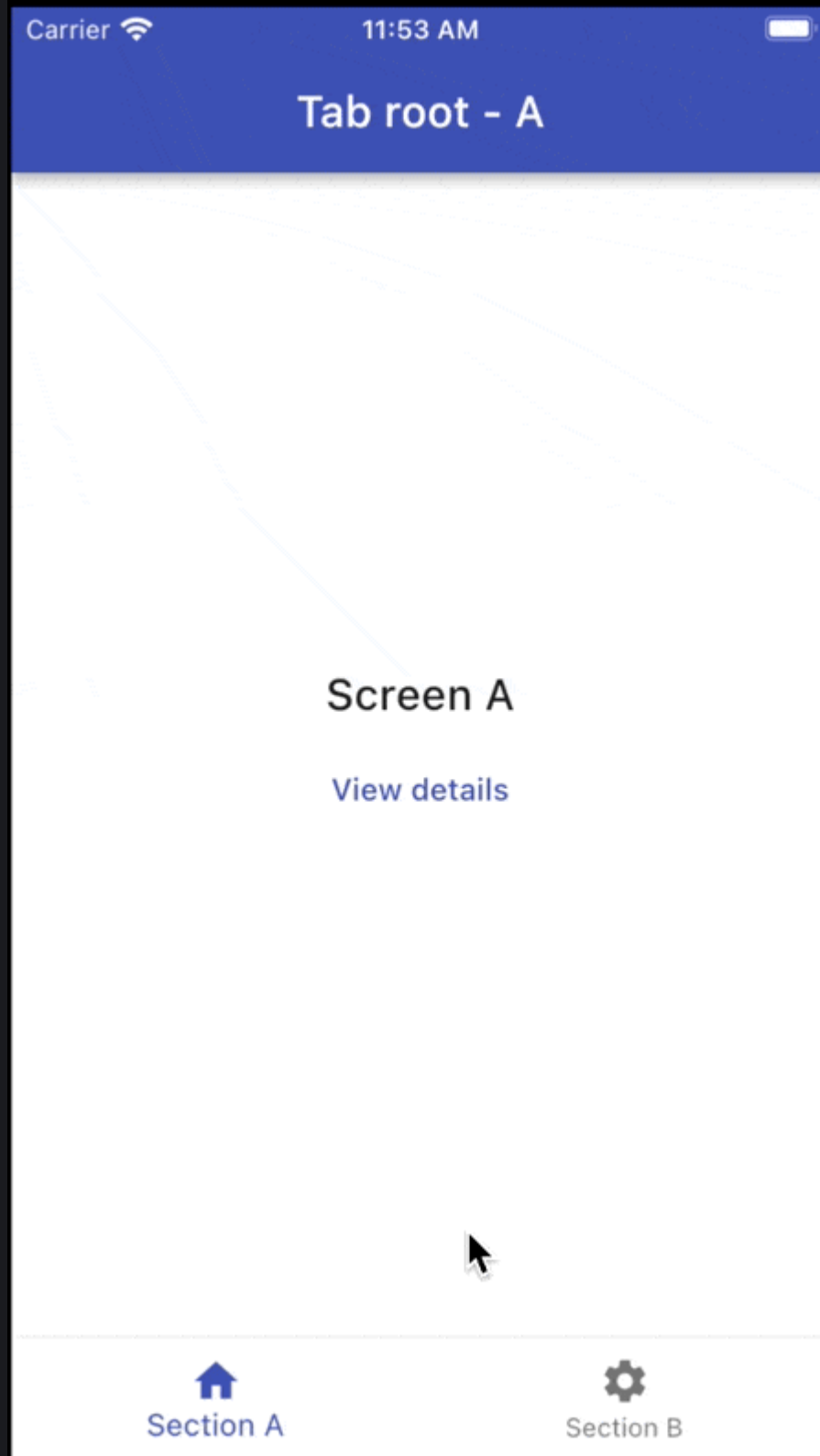
Responsive UI that uses a NavigationBar on mobile and a NavigationRail on wider screens

Finally, I'll share some reference source code you can use as a template so you don't have to reinvent the wheel in your Flutter apps.

Ready? Let's go!

Stateful Nested Navigation with GoRouter

Before we dive into the code, let's take another look at this:



Stateful nested navigation example. Note how the counter value is preserved when switching between tabs

As we can see, the UI above lets us switch between different pages when we click on the corresponding tabs (called “Section A” and “Section B”).

And to build this simple app, two separate features are required:

- **Support multiple navigation stacks:** offered by the `ShellRoute` API (available since GoRouter [4.5.0](#))

- **Preserve the state of routes:** offered by the `StatefulShellRoute`, `StatefulShellBranch`, and `StatefulNavigationShell` APIs (available since GoRouter 7.1.0)

So let's see how to use the new APIs to get the desired result.

GoRouter Implementation with StatefulShellRoute

According to the documentation, `StatefulShellRoute` is:

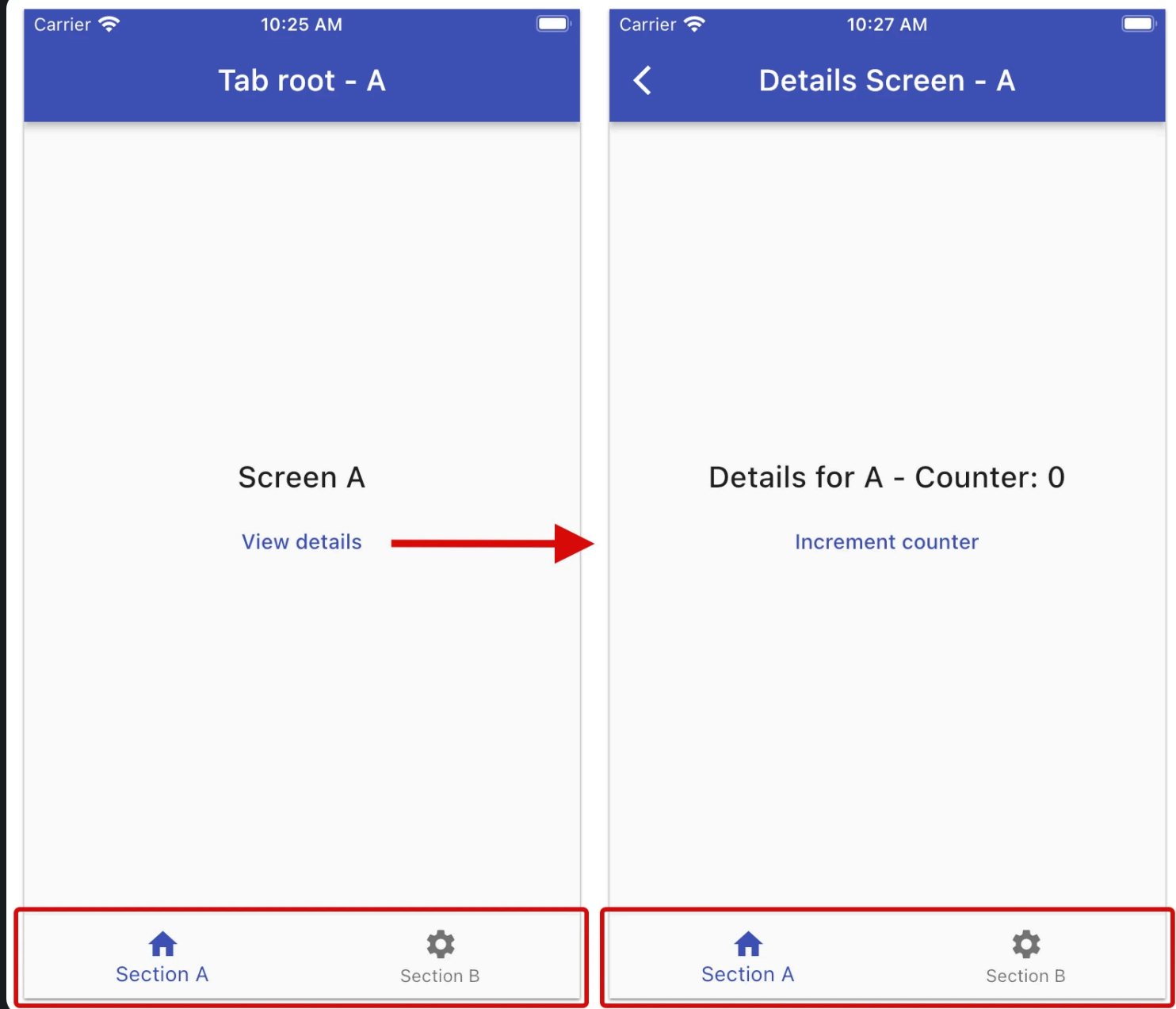
A route that displays a UI shell with separate Navigators for its sub-routes.

It also says this:

Similar to ShellRoute, this route class places its sub-route on a different Navigator than the root Navigator. However, this route class differs in that it creates separate Navigators for each of its nested branches (i.e. parallel navigation trees), making it possible to build an app with stateful nested navigation.

How does this work in practice?

Let's consider this example for reference:



NavigationBar example with two tabs

When the detail page is presented, it is stacked above Screen A, but the tabs at the bottom (the UI shell) remain visible.

The corresponding route hierarchy can be represented like this:

```
GoRouter
├─ StatefulShellRoute
│   ├── StatefulShellBranch
│   │   ├── GoRoute('/a')
│   │   └── GoRoute('details')
│   └── StatefulShellBranch
│       ├── GoRoute('/b')
│       └── GoRoute('details')
```

And implemented like this using the GoRouter APIs:

```
// private navigators
final _rootNavigatorKey = GlobalKey<NavigatorState>();
final _shellNavigatorAKey = GlobalKey<NavigatorState>(debugLabel: 'shellA')
final _shellNavigatorBKey = GlobalKey<NavigatorState>(debugLabel: 'shellB')

// the one and only GoRouter instance
final goRouter = GoRouter(
  initialLocation: '/a',
  navigatorKey: _rootNavigatorKey,
  routes: [
    // Stateful nested navigation based on:
    //
https://github.com/flutter/packages/blob/main/packages/go\_router/example/lib
    StatefulShellRoute.indexedStack(
      builder: (context, state, navigationShell) {
        // the UI shell
        return ScaffoldWithNestedNavigation(
          navigationShell: navigationShell);
      },
      branches: [
        // first branch (A)
        StatefulShellBranch(
          navigatorKey: _shellNavigatorAKey,
          routes: [
            // top route inside branch
            GoRoute(
              path: '/a',
              pageBuilder: (context, state) => const NoTransitionPage(
                child: RootScreen(label: 'A', detailsPath: '/a/details'),
              ),
              routes: [
                // child route
                GoRoute(
                  path: 'details',
                  builder: (context, state) =>
                    const DetailsScreen(label: 'A'),
                ),
              ],
            ),
          ],
        ),
        // second branch (B)
        StatefulShellBranch(
          navigatorKey: _shellNavigatorBKey,
```

```
routes: [
    // top route inside branch
    GoRoute(
        path: '/b',
        pageBuilder: (context, state) => const NoTransitionPage(
            child: RootScreen(label: 'B', detailsPath: '/b/details'),
        ),
        routes: [
            // child route
            GoRoute(
                path: 'details',
                builder: (context, state) =>
                    const DetailsScreen(label: 'B'),
            ),
        ],
    ),
],
),
],
),
],
),
];

// use like this:
// MaterialApp.router(routerConfig: goRouter, ...)
```

A few notes:

- The `GoRouter` instance should be declared as a **global variable** so it doesn't get rebuilt on hot reload.
- We use `StatefulShellRoute.indexedStack` to create a custom `ScaffoldWithNestedNavigation` widget (defined below), which takes a `StatefulNavigationShell` argument.
- `StatefulShellRoute` takes a List of `StatefulShellBranch` items, each representing a separate stateful branch. in the route tree.
- Both `GoRouter` and `StatefulShellBranch` take a `navigatorKey` argument (all navigators are defined at the top).
- We use a `NoTransitionPage` inside the `/a` and `/b` routes to prevent unintended animations when switching between tabs (this is the default behaviour on popular iOS apps).

- Each `StatefulShellBranch` can define its own route hierarchy (using the `GoRoute` API we're already familiar with).
- `RootScreen` and `DetailsScreen` are just simple widgets with a `Scaffold` and a regular `AppBar`.

What's more interesting is the builder argument inside

`StatefulShellRoute.indexedStack`. 📌

Where the magic happens: StatefulNavigationShell

When we use `StatefulShellRoute.indexedStack`, we get a `builder` that gives us a `navigationShell` we can use to construct our shell:

```
StatefulShellRoute.indexedStack(
  builder: (context, state, navigationShell) {
    // the UI shell
    return ScaffoldWithNestedNavigation(navigationShell:
navigationShell);
  },
  branches: [ ... ]
)
```

Here's how we may implement this class:

```
// Stateful nested navigation based on:
//
// https://github.com/flutter/packages/blob/main/packages/go_router/example/lib/
class ScaffoldWithNestedNavigation extends StatelessWidget {
  const ScaffoldWithNestedNavigation({
    Key? key,
    required this.navigationShell,
  }) : super(key: key ?? const ValueKey('ScaffoldWithNestedNavigation'));
  final StatefulNavigationShell navigationShell;

  void _goBranch(int index) {
    navigationShell.goBranch(
      index,
      // A common pattern when using bottom navigation bars is to support
      // navigating to the initial location when tapping the item that is
      // already active. This example demonstrates how to support this behavior
      // using the initialLocation parameter of goBranch.
```

```

        initialLocation: index == navigationShell.currentIndex,
    );
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        body: navigationShell,
        bottomNavigationBar: NavigationBar(
            selectedIndex: navigationShell.currentIndex,
            destinations: const [
                NavigationDestination(label: 'Section A', icon: Icon(Icons.home)),
                NavigationDestination(label: 'Section B', icon: Icon(Icons.settings)),
            ],
            onDestinationSelected: _goBranch,
        ),
    );
}

```

As we can see, the `navigationShell` is passed directly to the `body` of the `Scaffold`:

```

Scaffold(
    body: navigationShell,
    bottomNavigationBar: NavigationBar(
        selectedIndex: navigationShell.currentIndex,
        destinations: const [
            NavigationDestination(label: 'Section A', icon: Icon(Icons.home)),
            NavigationDestination(label: 'Section B', icon:
Icon(Icons.settings)),
        ],
        onDestinationSelected: _goBranch,
    ),
)

```

And we can also use it to:

- retrieve the `currentIndex` that we can pass to the `NavigationBar`
- call the `goBranch` method so we can switch to a new branch when a new destination is selected

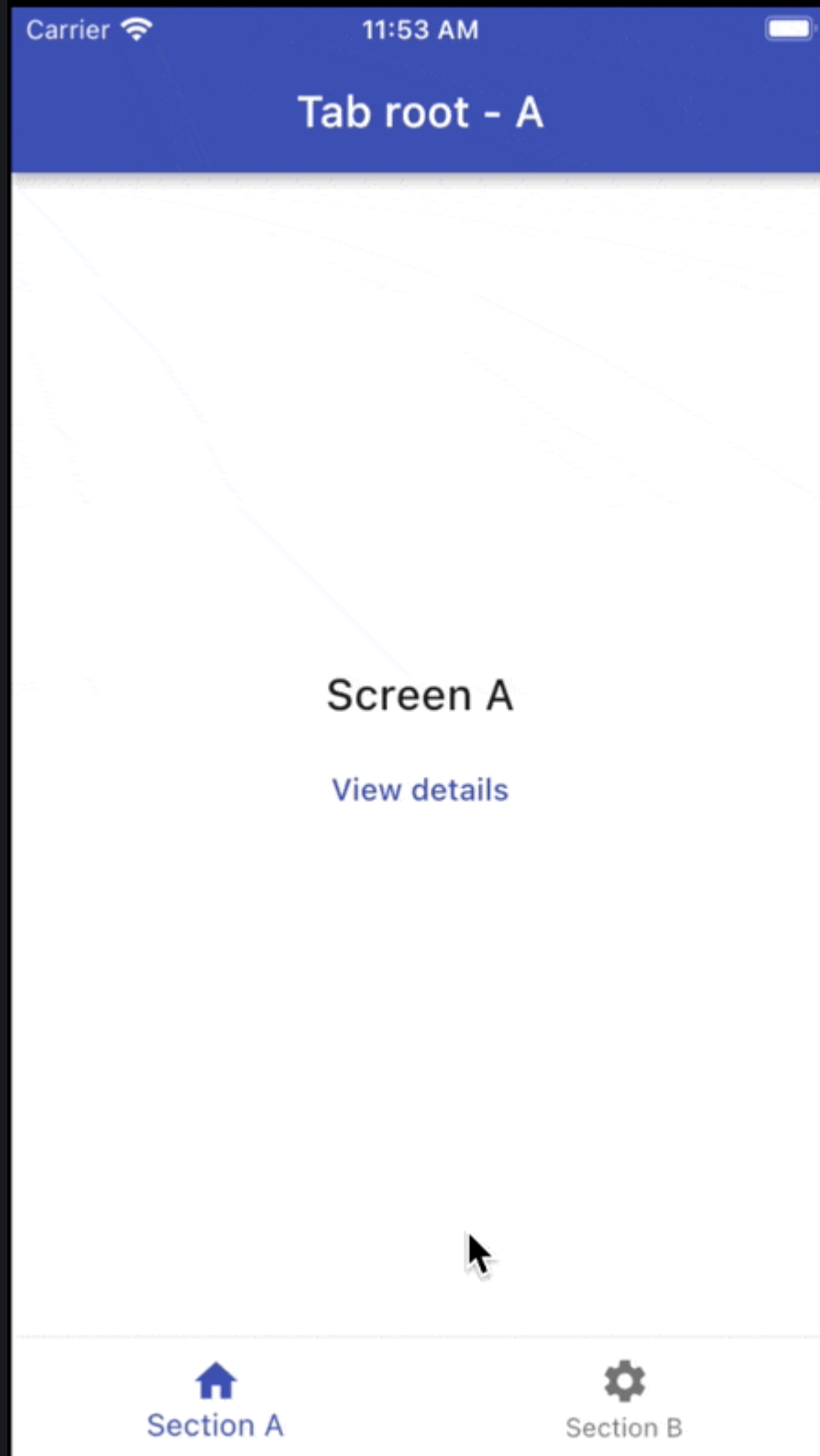
What's great about using `navigationShell` is that we don't have to store and update any state (such as the selected index) inside our `ScaffoldWithNestedNavigation` (which is a `StatelessWidget`). That's because all the stateful logic lives inside the `StatefulNavigationShell` class itself - a widget that manages the state of a `StatefulShellRoute`, by creating a separate `Navigator` for each of its nested branches.

Stateful Nested Navigation: Summary

As it turns out, the code above already solves these problems:

- We can easily switch between tabs (using `navigationShell.goBranch`) by writing only stateless code
- Stateful nested navigation “just works”, and each branch remembers its own state (all the pages inside each tab)

Here's what the end result looks like:



Stateful nested navigation example. Note how the counter value is preserved when switching between tabs

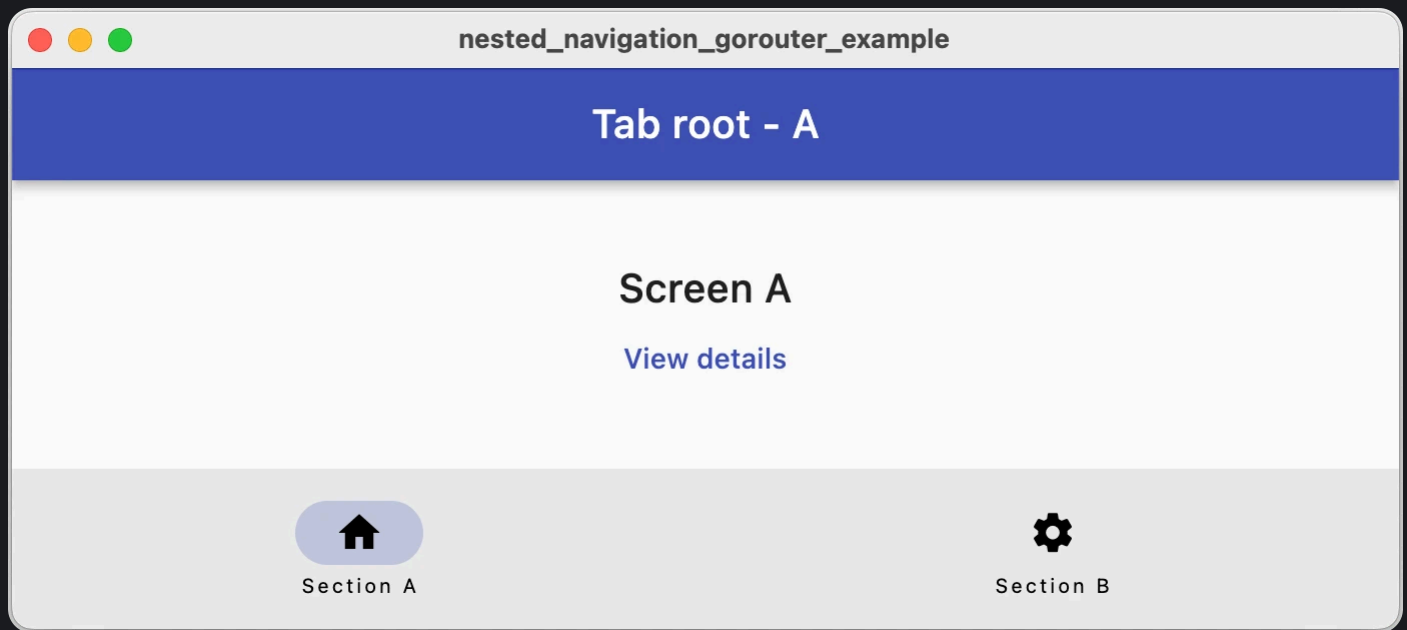
To make this possible, we only needed a few ingredients:

- A `StatefulShellRoute` which defines a list of branches
- A custom `ScaffoldWithNestedNavigation` widget that declares the desired shell UI using a `NavigationBar`
- The logic for switching between branches with the `navigationShell` object

And now that we've covered the core functionality, let's make our UI responsive. 📌

Making the UI responsive with NavigationRail and LayoutBuilder

Bottom navigation with `NavigationBar` works well on mobile but not so much on bigger form factors:

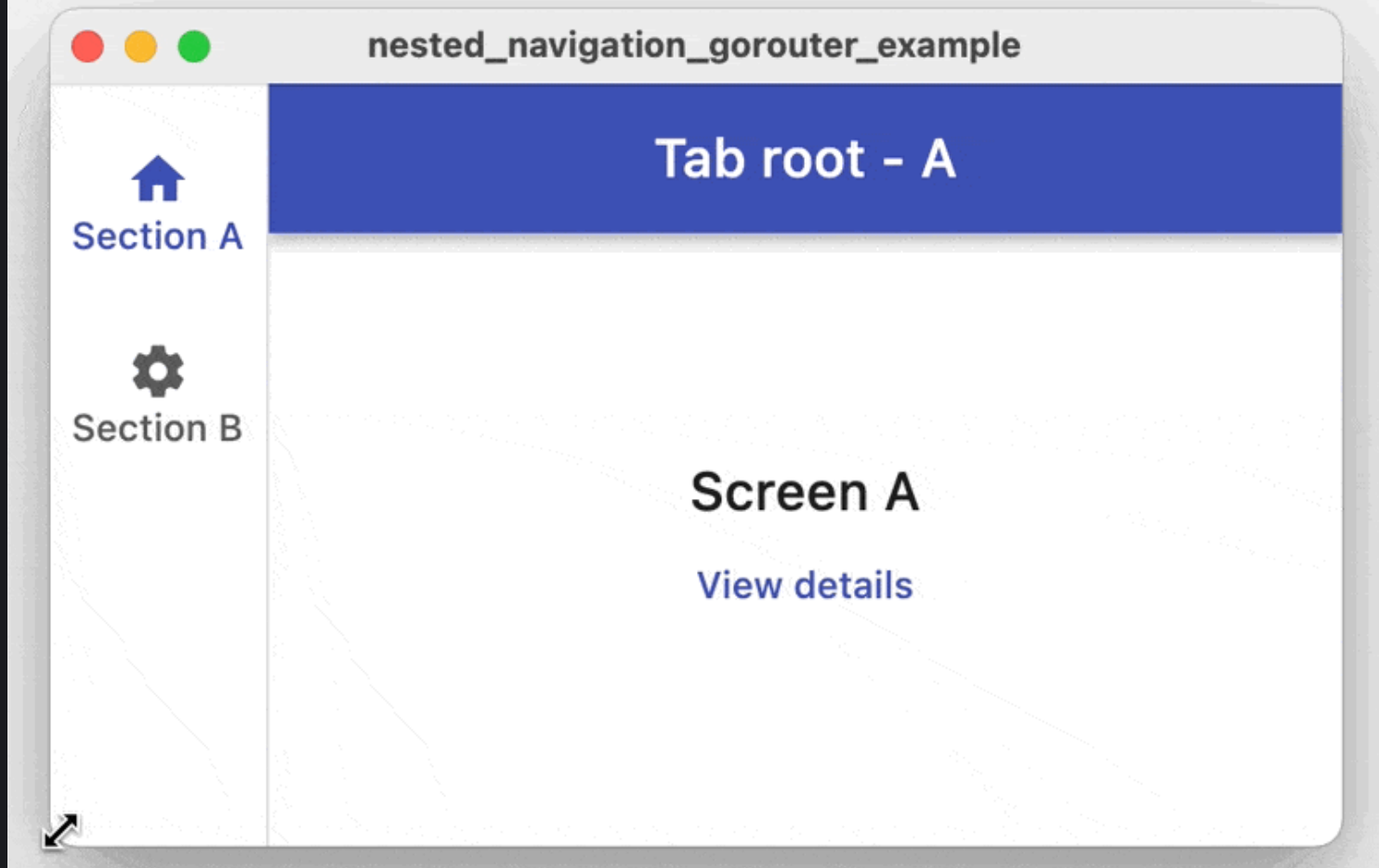


A bottom navigation bar applied to a widescreen layout

While we can use packages such as [flutter_adaptive_scaffold](#) to create complex responsive layouts, this is overkill for our simple example.

Instead, we can leverage `LayoutBuilder` and `NavigationRail` to make our UI responsive without any 3rd party packages.

Here's what we're after:



Responsive UI that uses a `NavigationBar` on mobile and a `NavigationRail` on wider screens

Looks good. Now let's go and build it! 📌

Creating a Scaffold with `NavigationBar` Widget

As a first step, let's move all the UI code from the `ScaffoldWithNestedNavigation` to a new `ScaffoldWithNavigationBar` widget:

```
class ScaffoldWithNavigationBar extends StatelessWidget {
  const ScaffoldWithNavigationBar({
    super.key,
    required this.body,
    required this.selectedIndex,
    required this.onDestinationSelected,
  });
  final Widget body;
  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: body,
```

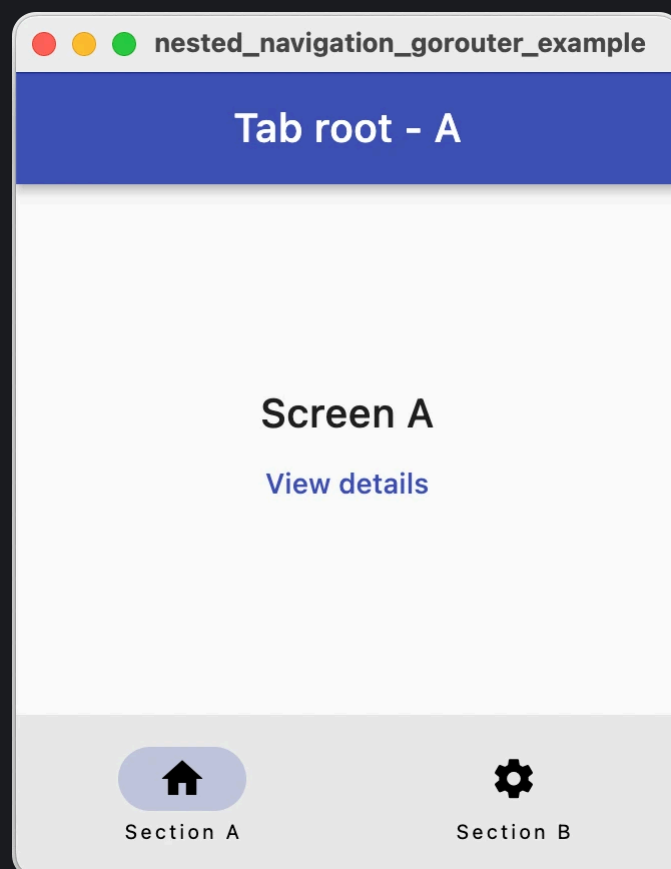
```

    bottomNavigationBar: NavigationBar(
      selectedIndex: selectedIndex,
      destinations: const [
        NavigationDestination(label: 'Section A', icon:
Icon(Icons.home)),
        NavigationDestination(label: 'Section B', icon:
Icon(Icons.settings)),
      ],
      onDestinationSelected: onDestinationSelected,
    ),
  );
}
}

```

Nothing fancy here. All this widget does is return a `Scaffold` with a `NavigationBar`, configured with the `body`, `selectedIndex`, and `onDestinationSelected` arguments that are passed to the constructor.

We'll use this widget on mobile only:



On narrow screens, we keep the navigation tabs at the bottom with a `NavigationBar` widget

Creating a Scaffold with `NavigationRail` Widget

But on wider screens, we need a different widget that uses `NavigationRail` instead:

```

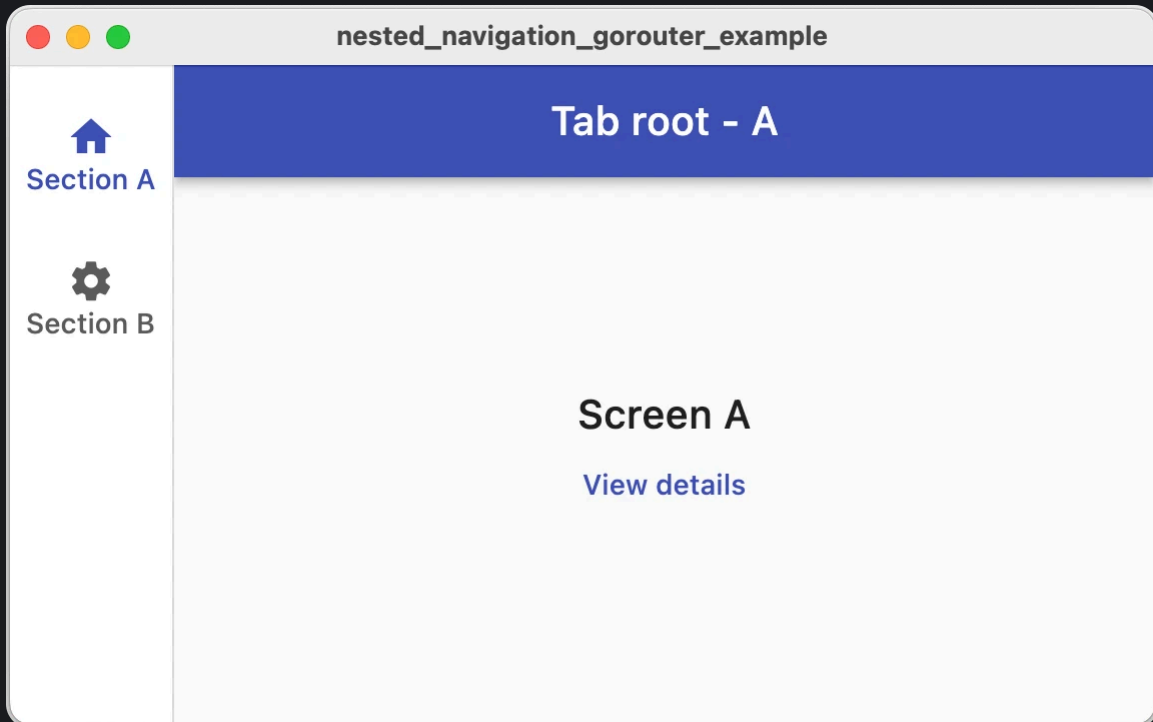
class ScaffoldWithNavigationRail extends StatelessWidget {
  const ScaffoldWithNavigationRail({
    super.key,
    required this.body,
    required this.selectedIndex,
    required this.onDestinationSelected,
  });
  final Widget body;
  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          // Fixed navigation rail on the left (start)
          NavigationRail(
            selectedIndex: selectedIndex,
            onDestinationSelected: onDestinationSelected,
            labelType: NavigationRailLabelType.all,
            destinations: const [
              NavigationRailDestination(
                label: Text('Section A'),
                icon: Icon(Icons.home),
              ),
              NavigationRailDestination(
                label: Text('Section B'),
                icon: Icon(Icons.settings),
              ),
            ],
          ),
          const VerticalDivider(thickness: 1, width: 1),
          // Main content on the right (end)
          Expanded(
            child: body,
          ),
        ],
      ),
    );
  }
}

```

Note how the `body` of the `Scaffold` is a `Row` with three children:

- a `NavigationRail`
- a `VerticalDivider`
- an `Expanded` widget that fills the rest of the screen with the `body` widget that is given as an argument.



On wide screens, we can move the navigation tabs to the side with a `NavigationRail` widget

With these widgets in place, we can update our `ScaffoldWithNestedNavigation` widget. 🙌

Updated Scaffold with Nested Navigation

As a last step, let's update the `build` method of our `ScaffoldWithNestedNavigation`:

```
class ScaffoldWithNestedNavigation extends StatelessWidget {
  const ScaffoldWithNestedNavigation({
    Key? key,
    required this.navigationShell,
  }) : super(key: key ?? const ValueKey<String>(
    'ScaffoldWithNestedNavigation'));
  final StatefulNavigationShell navigationShell;

  void _goBranch(int index) { ... }

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(builder: (context, constraints) {
      // layout breakpoint: tweak as needed
```

```

    if (constraints.maxWidth < 450) {
      return ScaffoldWithNavigationBar(
        body: navigationShell,
        selectedIndex: navigationShell.currentIndex,
        onDestinationSelected: _goBranch,
      );
    } else {
      return ScaffoldWithNavigationRail(
        body: navigationShell,
        selectedIndex: navigationShell.currentIndex,
        onDestinationSelected: _goBranch,
      );
    }
  });
}
}

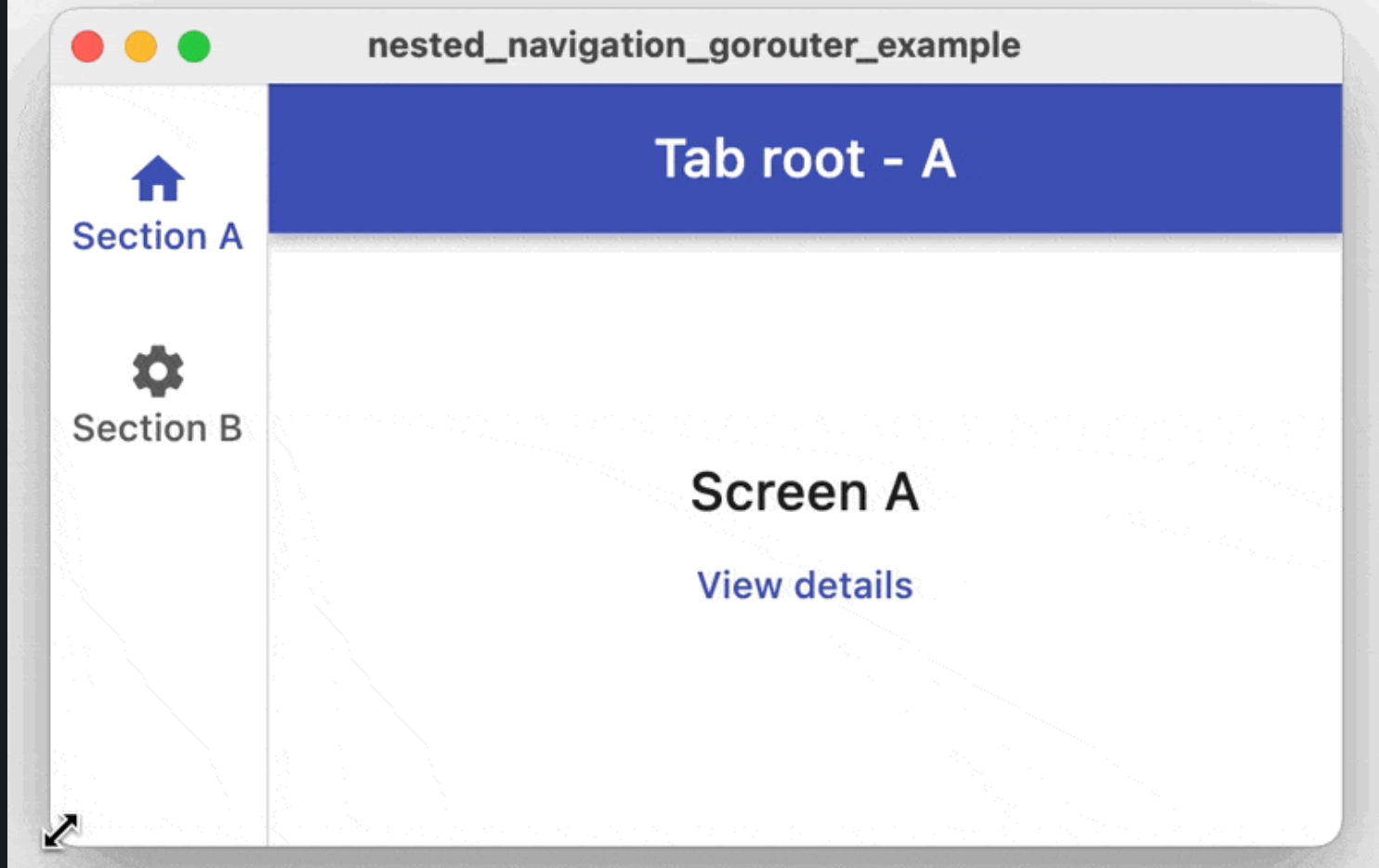
```

Note how we use a `LayoutBuilder` to decide which widget to return, depending on the maximum width of the parent widget.

As an alternative to `LayoutBuilder`, we can obtain the size from `MediaQuery.sizeOf(context)`. Either way, our widget will rebuild if the parent size changes (for example, when resizing the window on web or desktop).

And since we have a good separation of concerns between UI and routing logic, we don't need to change anything in the routing code.

And voilà! With these changes, our UI is now responsive:



Responsive UI that uses a `NavigationBar` on mobile and a `NavigationRail` on wider screens

Source Code

In this article, I have only focused on the important logic needed to enable stateful nested navigation.

You can find the complete source code on GitHub and use it as a template for your projects:

- [Stateful Nested Navigation examples: GoRouter vs Beamer](#)

I've also adopted the same approach in my Time Tracker app, which is a more complex project that uses Riverpod and a Firebase backend:

- [Time Tracking app with Flutter & Firebase](#)

Answering Common Questions

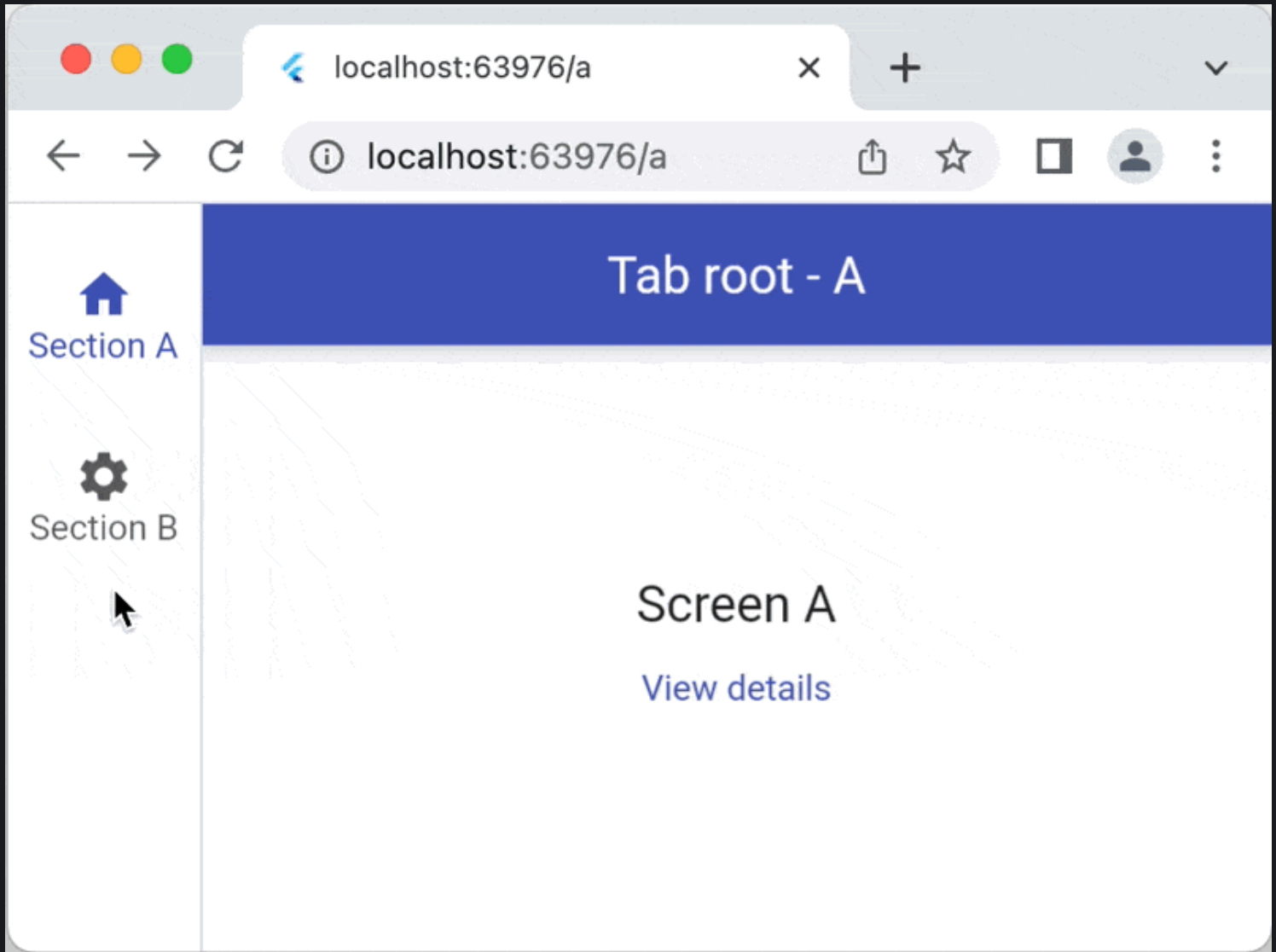
Before we wrap up, I want to discuss some use cases and issues you may encounter when working with stateful navigation. 📌

Does Stateful Nested Navigation work on Flutter web?

Absolutely!

I've tested both the example app and my time tracker app, and I can confirm that navigation by URL works as intended.

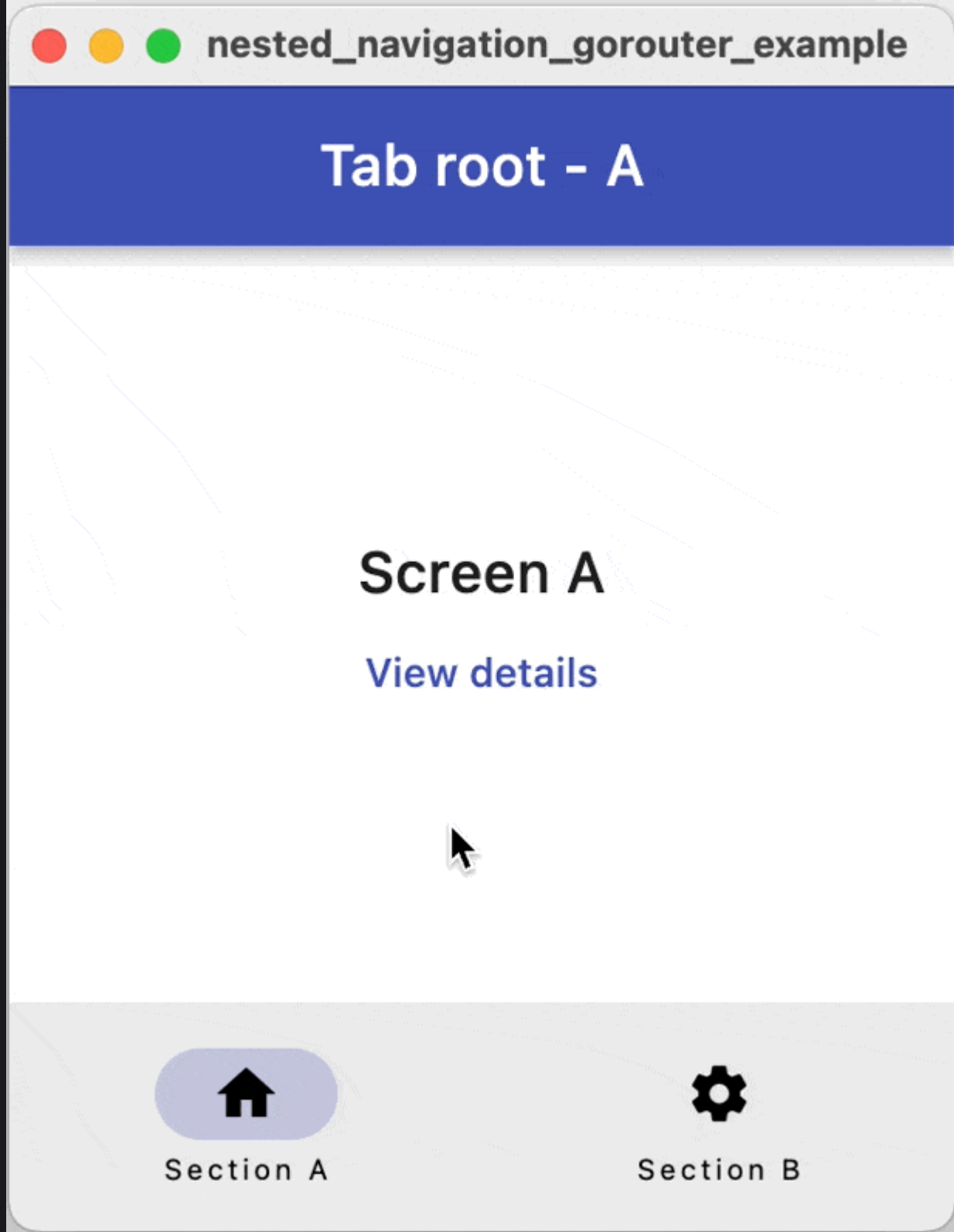
The browser forward and back buttons also work as expected (that is, the browser will correctly remember the history of opened pages, even when using stateful branches):



Stateful navigation on Flutter web: note how the URL is updated and the browser back button works as intended

Does “pop-to-root” work?

A common behaviour on mobile is that when we click on a bottom navigation tab **that is already selected**, we're taken back to the root of the corresponding stack of pages:



When clicking on a tab that is already selected, the app pop to the root of the corresponding navigator

This is exactly what the `initialLocation` argument of the `goBranch` method is for:

```
void _goBranch(int index) {  
  navigationShell.goBranch(  
    index,  
    // A common pattern when using bottom navigation bars is to support  
    // navigating to the initial location when tapping the item that is  
    // already active. This example demonstrates how to support this  
    behavior,
```

```
// using the initialLocation parameter of goBranch.  
initialLocation: index == navigationShell.currentIndex,  
);  
}
```

What About the Back Button on Android?

This is what I have observed when pressing the back button:

- If the currently selected branch has more than one route, the app will pop back to the parent route (works as expected).
- If there's only one route in the selected branch (and the selected branch itself is a top-level route), then the app will exit and go to the background (works as expected).

Details for B - Counter: 2

Increment counter



Section A



Section B



Back-button behaviour on Android

Note: apps based on Navigator 1.0 can use the `WillPopScope` widget to prevent back navigation based on some conditional logic. However, `WillPopScope` does not work with page-backed routes. For more info, read: [Replace redirect with onEnter and onExit](#).

Can I Navigate Programmatically to Another Branch from a Nested Route?

Easy peasy!

In fact, all you have to do is call this:

```
// go to the branch at the desired index
StatefulNavigationShell.of(context).goBranch(index);
```

Note that this will only work if you're **inside one of the subroutes** of the `StatefulShellRoute`.

For example, given the following route hierarchy:

```
GoRouter
├─ GoRoute('/about')
└─ StatefulShellRoute
    ├─ StatefulShellBranch
    │   └─ GoRoute('/a')
    │       └─ GoRoute('details')
    └─ StatefulShellBranch
        └─ GoRoute('/b')
            └─ GoRoute('details')
```

Then you can call `StatefulNavigationShell.of(context)` from the following routes:

- `/a`
- `/a/details`
- `/b`
- `/b/details`

But if you call it from `/about`, you'll get a `'shellState != null'` assertion.

For more information, check the `of` and `maybeOf` static methods of the `StatefulNavigationShell` class.

Are there some Unexpected Issues with GoRouter?

Ahem, yes. 😞

For example, at some point I bumped into this issue that was causing an exception when hot reload is triggered:

- [ShellRoute with CupertinoTabScaffold crashes with "multiple widgets used the same GlobalKey"](#)

Later on, I was told that this happens if the `GoRouter` instance is created inline. The solution is to declare `GoRouter` as a global variable (or inside a `Provider`), so it doesn't get rebuilt on hot reload. In other words, I was using it wrong - though this is an easy mistake and I hope a linter rule will be introduced to prevent this.

But to be fair, there are a bunch of other [GoRouter issues](#) on GitHub, and I hope they will be resolved soon.

Which Routing Package Should You Use?

Navigation is a complex topic and any Flutter routing package worth its salt needs to support many features across all supported platforms (mobile, desktop, and web).

So it's not surprising that even the most popular packages have dozens of open issues:

- [GoRouter issues](#)
- [AutoRoute issues](#)
- [Beamer issues](#)

And while they're not perfect, these packages are very valuable and can save you a ton of time, compared to rolling out your own routing code.

So pick one and use it. And if it works well, show appreciation to the contributors that maintain it (often for free). 🙏

Conclusion

I've been following the evolution of Flutter's navigation APIs for a few years now.

From Navigator 1.0 to 2.0 and all the [3rd party packages that have been created](#), it's been a bumpy road.

When GoRouter came out, it quickly became the most loved routing package, but I and others quickly realised its limitations.

So I've been patiently waiting for stateful nested navigation to land since [this PR was first opened in September 2022](#). And I must say I have the [highest admiration](#) for how [Tobias Löfstrand](#) carried this [complex piece of work](#) from beginning to end over many rounds of feedback, making GoRouter more valuable than ever before. 🙏

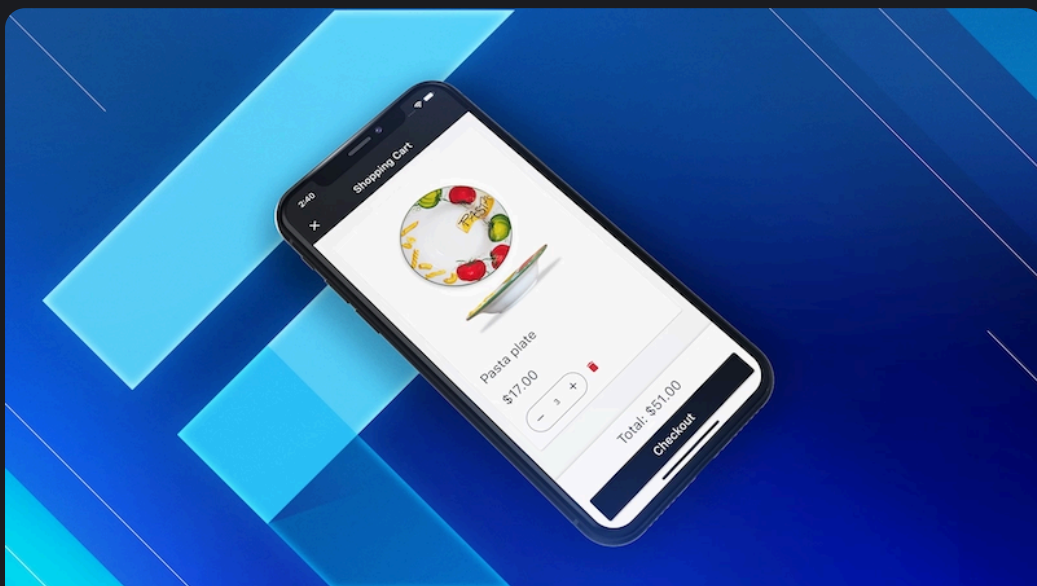
In fact, he even contributed [some example code](#) that made my job easy when writing this article. And I hope that by shining some light on his work, I've helped you understand how to use the new GoRouter stateful navigation APIs in your projects.

And while GoRouter still has [its quirks](#), I hope it will continue to improve and become a robust (and well documented) routing package we can all rely on. 🙌

Happy coding!

Flutter Foundations Course Now Available

I launched a brand new course that covers GoRouter in great depth, along with other important topics like state management, app architecture, testing, and much more:



Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

Invest in yourself with my high-quality Flutter courses.



Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀



Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase.



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

[Terms of Use](#)